

Playwright-Notes-8

Annotate Test in Playwright | Playwright Tutorials

- Annotation is a way of adding some extra information into the test.
- If you would like to annotate your tests with something more substantial than a tag, you can do that when declaring a test.
- Annotations have a type and a description for more context, and will be visible in the test report.

Syntax -

```
import { test, expect } from '@playwright/test';

test('test login page', {
  annotation: {
    type: 'issue',
    description: 'https://github.com/microsoft/playwright/issues/23180',
  },
}, async ({ page }) => {
  // ...Test Script Code
});
```

In VSCode:

Create a file called `AnnotateTestPractice.spec.ts` and include the following code:

```
import { test, expect } from "@playwright/test";

test("Annotate Test Practice Test 1", {
  annotation: {
    type: "Jira Story",
    description: "https://wishinfinite1.atlassian.net/jira/software/projects/SCRUM/boards/1/backlog?selectedIssue=SCRUM-1"
  }
}, async ({ page }) => {
  await page.goto(`https://www.google.com`);
  await expect(page).toHaveTitle("Google");
})

test.skip("Annotate Test Practice Test 2", {
  tag: "@UI",
  annotation: [{
    type: "Skip Reason",
    description: "Requirement change is going to happen"
  }, {
    type: "Jira Story",
```

```

description:
"https://wishinfinite1.atlassian.net/jira/software/projects/SCRUM
/boards/1/backlog?selectedIssue=SCRUM-1"
    }],

}, async ({ page }) => {
    await page.goto(`https://www.google.com`);
    await expect(page).toHaveTitle("Google");
})

test.describe("Describe Block 1", {
    annotation: {
        type: "Jira Story",
description:
"https://wishinfinite1.atlassian.net/jira/software/projects/SCRUM
/boards/1/backlog?selectedIssue=SCRUM-1"
    }
}, async () => {

    test("Practice Test1", async ({ }) => {
        console.log("Practice Test 1")
    })

    test("Practice Test2", async ({ }) => {
        console.log("Practice Test 2")
    })

    test("Practice Test3", async ({ }) => {
        console.log("Practice Test 3")
    })
})

```

Explanation of code:

Code Explanation

This file demonstrates annotations in Playwright — how to attach metadata like Jira links and skip reasons to tests and describe blocks.

Import

```
import { test, expect } from "@playwright/test";
```

- Imports both **test** and **expect** since this file has assertions
-

Test 1 — Single Annotation on a Test

```
test("Annotate Test Practice Test 1", {
  annotation: {
    type: "Jira Story",
    description: "https://wishinfinite1.atlassian.net/..."
  }
}, async ({ page }) => {
  await page.goto(`https://www.google.com`);
  await expect(page).toHaveTitle("Google");
})
```

Breaking It Down

```
test("test name", { options }, async ({ page }) => { ... })
//      ↑
//      This is where annotation goes
```

Part	Detail
annotation :	The annotation option key
type: "Jira Story"	A label for the annotation — can be anything meaningful
description:	The value — here it's a Jira ticket URL
toHaveTitle("Google")	Asserts page title is exactly "Google"

How It Appears in Reports

✓ Annotate Test Practice Test 1

Test 2 — Skip + Multiple Annotations

```
test.skip("Annotate Test Practice Test 2", {
  tag: "@UI",
  annotation: [{
    type: "Skip Reason",
    description: "Requirement change is going to happen"
  }, {
    type: "Jira Story",
    description: "https://wishinfinite1.atlassian.net/..."
  }],
}, async ({ page }) => {
  await page.goto(`https://www.google.com`);
  await expect(page).toHaveTitle("Google");
})
```

Breaking It Down

```
test.skip()
test.skip("test name", { options }, async () => { ... })
// ↑
// test is SKIPPED — body never executes
```

- Test is skipped but still appears in the report
- Annotations still show up explaining WHY it was skipped

Tag + Annotations Together

```
{
  tag: "@UI",          // ← for filtering tests via CLI
  annotation: [...]    // ← for report metadata
}
```

- **tag** and **annotation** can coexist in the same options object
- They serve different purposes:

Option	Purpose	Used For
<code>tag</code>	Filter/group tests	CLI <code>--group @UI</code>
<code>annotation</code>	Attach metadata	HTML reports

Multiple Annotations (Array)

```
annotation: [
  {
    type: "Skip Reason",      // annotation 1
    description: "Requirement change is going to happen"
  },
  {
    type: "Jira Story",      // annotation 2
    description: "https://..."
  }
]
```

- Single annotation → plain object `{ type, description }`
- Multiple annotations → array of objects `[{...}, {...}]`

How It Appears in Reports

 Annotate Test Practice Test 2 [SKIPPED] [@UI]
 Skip Reason : Requirement change is going to happen
 Jira Story : [https://wishinfinite1.atlassian.net/...](https://wishinfinite1.atlassian.net/)

Test 3 — Annotation on `describe` Block

```
test.describe("Describe Block 1", {
  annotation: {
    type: "Jira Story",
    description: "https://wishinfinite1.atlassian.net/..."
  }
})
```

```

    }
  }, async () => {
    test("Practice Test1", async () => {
      console.log("Practice Test 1")
    })
    test("Practice Test2", async () => {
      console.log("Practice Test 2")
    })
    test("Practice Test3", async () => {
      console.log("Practice Test 3")
    })
  })
})

```

Breaking It Down

- Annotation is placed on the describe block instead of individual tests
- All tests inside inherit the annotation
- Only one Jira link needed — applies to the whole group

How It Appears in Reports

Describe Block 1

 Jira Story: <https://wishinfinite1.atlassian.net/...>

- ├──  Practice Test1
- ├──  Practice Test2
- └──  Practice Test3

Single vs Multiple Annotations Syntax

// Single annotation — plain object

```

annotation: {
  type: "Jira Story",
  description: "https://..."
}

```

// Multiple annotations — array of objects

```

annotation: [
  { type: "Skip Reason", description: "Requirement change" },
  { type: "Jira Story", description: "https://..." },
  { type: "Owner", description: "John" },
  { type: "Severity", description: "Critical" }
]

```

]

Where Annotations Can Be Placed

`test()`  on individual test
`test.skip()`  on skipped test
`test.fail()`  on expected fail test
`test.fixme()`  on fixme test
`test.describe()`  on entire group

All Three Tests Summary

Test	Annotation Type	Skipped?
Test 1	Single — Jira Story	 Runs
Test 2	Multiple — Skip Reason + Jira Story	 Skipped
Describe Block	Single — Jira Story (inherited by all)	 Runs

Custom Annotation Types You Can Use

// There are NO fixed type names — use anything meaningful
annotation: [
 { type: "Jira Story", description: "SCRUM-123" },
 { type: "Bug", description: "BUG-456" },
 { type: "Skip Reason", description: "Feature not ready" },
 { type: "Owner", description: "QA Team" },
 { type: "Severity", description: "Critical" },

```
{ type: "Environment", description: "Staging only" },  
{ type: "Feature",    description: "Login Module" }  
]
```

Key Takeaways

Concept	Detail
annotation	Attaches metadata to tests for reports
type	Label for the annotation — any string
description	Value/detail — URL, text, ticket number
Single annotation	Plain object { type , description }
Multiple annotations	Array of objects [{...}, {...}]
On describe block	All child tests inherit the annotation
Combined with tag	tag and annotation can coexist
Visible in	Playwright HTML reports

Best practice — always annotate tests with Jira/ticket links so anyone reading the report can immediately trace a failure back to its requirement or bug report.

Annotations in Playwright

What Are Annotations? (Layman Terms)

Think of annotations like sticky notes you put on your tests to give them special instructions or extra information.

Just like you might put a sticky note on a file saying:

-  "Skip this for now"
-  "This is broken, don't run"
-  "This has a known bug"
-  "See ticket JIRA-123 for details"

Annotations tell Playwright how to treat a test without deleting or changing the test itself.

Types of Annotations

1. `test.skip()` — Skip This Test

```
// Skip always
test("Login Test", async ({ page }) => {
  test.skip();
  // test won't run at all
})

// Skip conditionally
test("Login Test", async ({ page }) => {
  test.skip(process.env.CI === "true", "Skipping on CI environment");
})
```

Layman: "Don't run this test right now, but keep it around"

Without skip → Test runs normally 

With skip → Test is ignored 

2. `test.fail()` — Expect This Test to Fail

```
test("Known Broken Test", async ({ page }) => {
  test.fail();
  await page.goto("https://example.com");
  await expect(page.locator("#broken-element")).toBeVisible();
})
```

```
})
```

Layman: "I know this test will fail — mark it as expected failure, don't alarm anyone"

Normal failing test →  FAIL (alerts the team)
test.fail() test →  PASS (expected to fail, no alarm)
test.fail() passes →  FAIL (unexpectedly passed — needs attention)

3. **test.fixme()** — This Needs Fixing

```
test("Broken Feature Test", async ({ page }) => {  
  test.fixme();  
  // test is skipped but marked as needs fixing  
})
```

// With condition

```
test("Broken Feature Test", async ({ page }) => {  
  test.fixme(browserName === "firefox", "Broken on Firefox, fix  
pending");  
})
```

Layman: "This test is broken and someone needs to fix it — flag it clearly"

test.skip() → "Skip quietly"
test.fixme() → "Skip BUT remind us to fix this!"

4. **test.slow()** — This Test Takes Long

```
test("Heavy Data Processing Test", async ({ page }) => {  
  test.slow();  
  // Playwright multiplies the default timeout by 3  
  await page.goto("https://slow-website.com");  
})
```

Layman: "Give this test extra time — it's a slow one"

Default timeout → 30 seconds

`test.slow()` timeout → 90 seconds (3x multiplier)

5. `test.info().annotations` — Custom Info Labels

```
test("Payment Test", async ({ page }) => {
  test.info().annotations.push(
    { type: "issue", description: "https://jira.com/PAYMENT-123" },
    { type: "severity", description: "critical" },
    { type: "owner", description: "John" }
  )
  // your test steps here
})
```

Layman: "Attach extra notes/links to the test for reports"

- Shows up in HTML test reports
 - Links tests to bug tickets or documentation
 - Adds metadata like severity, owner, feature name
-

6. `test.describe.skip()` — Skip Entire Group

```
test.describe.skip("Payment Tests", () => {
  test("Credit Card", async ({ page }) => { ... })
  test("PayPal", async ({ page }) => { ... })
  test("Apple Pay", async ({ page }) => { ... })
  // ALL tests skipped
})
```

Layman: "Skip the whole folder of tests, not just one"

7. `test.describe.fixme()` — Fix Entire Group

```
test.describe.fixme("Broken Feature Tests", () => {
  test("Test 1", async ({ page }) => { ... })
  test("Test 2", async ({ page }) => { ... })
})
```

```
// ALL marked as fixme
})
```

All Annotations Side by Side

Your Test Suite

test.skip()	→		Skipped (not run)
test.fail()	→		Expected to fail (no alarm)
test.fixme()	→		Skipped + flagged for fixing
test.slow()	→		Extra timeout given (3x)
annotations	→		Extra info in reports

Real World Analogy

Annotation	Real World Equivalent
test.skip()	"Out of Office" — not available right now
test.fail()	"Expected to be late" — no surprise when it happens
test.fixme()	"On the TODO list" — broken, needs attention
test.slow()	"Takes longer than usual" — be patient

annotations "Sticky note with extra info" — for reference

Conditional Annotations (Most Useful)

```
test("Browser Specific Test", async ({ page, browserName }) => {  
  test.skip(browserName === "webkit", "Not supported on Safari");  
  test.slow(browserName === "firefox", "Firefox runs slower");  
  test.fail(browserName === "chrome", "Known bug on Chrome");  
})
```

- Annotations can be conditional based on:
 - Browser type
 - Environment (CI/local)
 - Operating system
 - Any custom condition
-

Key Takeaways

Annotation	What it does	Test Runs?
test.skip()	Ignores the test	 No
test.fail()	Expects failure	 Yes
test.fixme()	Flags for fixing	 No
test.slow()	Extends timeout	 Yes

annotations.push()

Adds report metadata

 **Yes**

Golden Rule — Annotations don't change what your test does, they change how Playwright treats your test when running and reporting.

Visual Testing in Playwright | Visually Compare Screenshot | Playwright Tutorials

- Playwright Test includes the ability to produce and **visually compare** screenshots using `await expect(page).toHaveScreenshot()`.
- On first execution, Playwright test will generate reference screenshots. Subsequent runs will compare against the reference.
- When you run the test for the first time, the test runner will say: Error: A snapshot doesn't exist at <Folder name>/<FileName>.png, writing actual.
- Playwright Test uses the [pixelmatch](#) library which is pixel-level image comparison library.

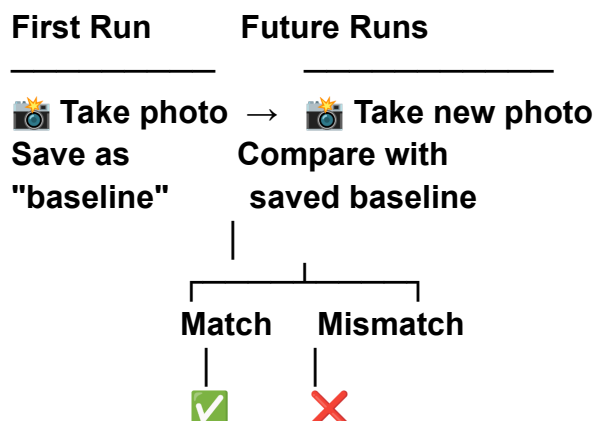
Options -

- fullPage
 - maxDiffPixels
 - maxDiffPixelRatio
 - mask
 - stylePath
-
- Apart from screenshots, you can use `expect(value).toMatchSnapshot(snapshotName)` to compare text or arbitrary binary data.
 - To Update the reference screenshots use command - `npm run playwright test --update-snapshots`

Visual Testing in Playwright

What is Visual Testing? (Layman Terms)

Think of visual testing like taking a photograph of your webpage and comparing it to a reference photo every time you run tests.



PASS FAIL

Core Method — `toHaveScreenshot()`

```
import { test, expect } from "@playwright/test";

test("Visual Test - Full Page", async ({ page }) => {
  await page.goto("https://www.google.com");

  // Takes screenshot and compares with baseline
  await expect(page).toHaveScreenshot("google-homepage.png");
})
```

First Run vs Subsequent Runs

FIRST RUN

No baseline exists



Playwright takes screenshot



Saves as baseline image



Test FAILS first time
(baseline created, nothing to compare yet)

SUBSEQUENT RUNS

Baseline exists



Playwright takes new screenshot



Compares pixel by pixel





Types of Visual Snapshots

1. Full Page Screenshot

```
test("Full Page Visual Test", async ({ page }) => {  
  await page.goto("https://example.com");  
  
  // Captures the ENTIRE page  
  await expect(page).toHaveScreenshot("full-page.png", {  
    fullPage: true  
  });  
})
```

2. Element Screenshot

```
test("Element Visual Test", async ({ page }) => {  
  await page.goto("https://example.com");  
  
  // Captures ONLY a specific element  
  const header = page.locator("header");  
  await expect(header).toHaveScreenshot("header.png");  
})
```

3. Viewport Screenshot (Default)

```
test("Viewport Visual Test", async ({ page }) => {  
  await page.goto("https://example.com");  
  
  // Captures only what's VISIBLE on screen  
  await expect(page).toHaveScreenshot("viewport.png");  
})
```

Handling Small Differences — Threshold Options

Sometimes pixels differ slightly due to:

- Font rendering
- Anti-aliasing

- Browser differences

```
test("Visual Test with Threshold", async ({ page }) => {  
  await page.goto("https://example.com");  
  
  await expect(page).toHaveScreenshot("page.png", {  
  
    // Allow 10% of pixels to be different  
    maxDiffPixelRatio: 0.1,  
  
    // OR allow specific number of pixels to differ  
    maxDiffPixels: 100,  
  
    // Pixel color difference tolerance (0-1)  
    threshold: 0.2,  
  
    // Wait for animations to finish  
    animations: "disabled"  
  })  
})
```

Option	What it means
<code>maxDiffPixelRatio</code>	% of pixels allowed to differ (0.1 = 10%)
<code>maxDiffPixels</code>	Exact number of pixels allowed to differ
<code>threshold</code>	How different a single pixel can be (0=exact, 1=any)
<code>animations</code>	"disabled" stops animations before screenshot

Masking Dynamic Content

Some parts of a page change every visit (dates, ads, banners). You can mask them:

```
test("Visual Test with Mask", async ({ page }) => {  
  await page.goto("https://example.com");
```

```

await expect(page).toHaveScreenshot("page.png", {
  mask: [
    page.locator(".advertisement"), // mask ads
    page.locator(".current-date"),  // mask date
    page.locator(".user-avatar"),   // mask profile pic
  ]
})
})

```

Before Mask	After Mask
Header	Header
 Today	 ← masked
Content	Content
 Ad Here	 ← masked

Global Visual Config in `playwright.config.ts`

```
import { defineConfig } from "@playwright/test";
```

```

export default defineConfig({
  expect: {
    // Global threshold for all visual tests
    toHaveScreenshot: {
      maxDiffPixelRatio: 0.1,
      threshold: 0.2,
      animations: "disabled"
    }
  },

  // Where baseline screenshots are stored
  snapshotDir: "./snapshots",
})

```

Updating Baselines

When UI changes intentionally, update the baseline:

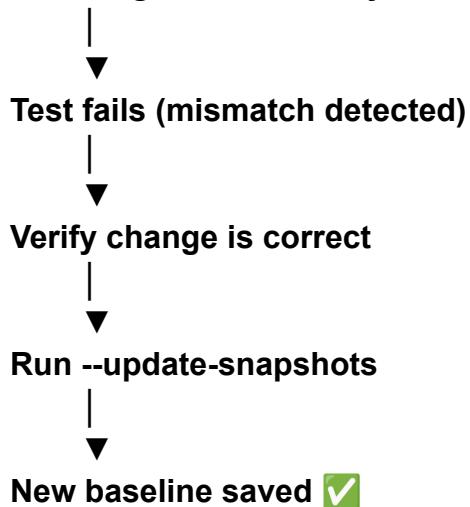
Update ALL baseline screenshots

`npx playwright test --update-snapshots`

Update specific test baseline

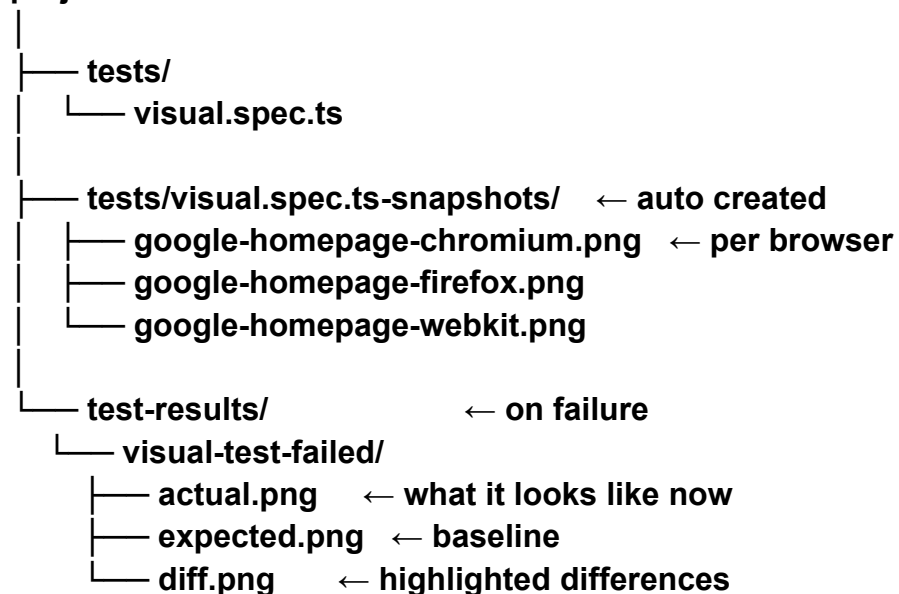
`npx playwright test visual.spec.ts --update-snapshots`

UI Changed Intentionally



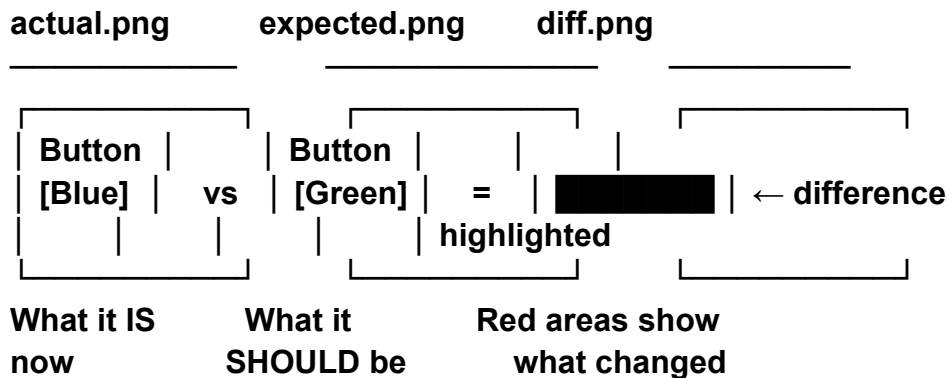
Where Screenshots Are Stored

project/



Failure Report — Diff Image

When a visual test fails, Playwright generates 3 images:



Complete Example

```
import { test, expect } from "@playwright/test";

test.describe("Visual Tests", () => {

  test("Homepage visual check", async ({ page }) => {
    await page.goto("https://example.com");

    // Disable animations for stable screenshots
    await page.evaluate(() => {
      document.body.style.animation = "none";
    });

    await expect(page).toHaveScreenshot("homepage.png", {
      fullPage: true,
      animations: "disabled",
      mask: [page.locator(".dynamic-banner")],
      maxDiffPixelRatio: 0.05
    });
  });

  test("Login form visual check", async ({ page }) => {
    await page.goto("https://example.com/login");

    // Check just the login form element
    const loginForm = page.locator(".login-form");
    await expect(loginForm).toHaveScreenshot("login-form.png", {
      threshold: 0.2
    });
  });
});
```

```
});  
})  
})
```

Key Takeaways

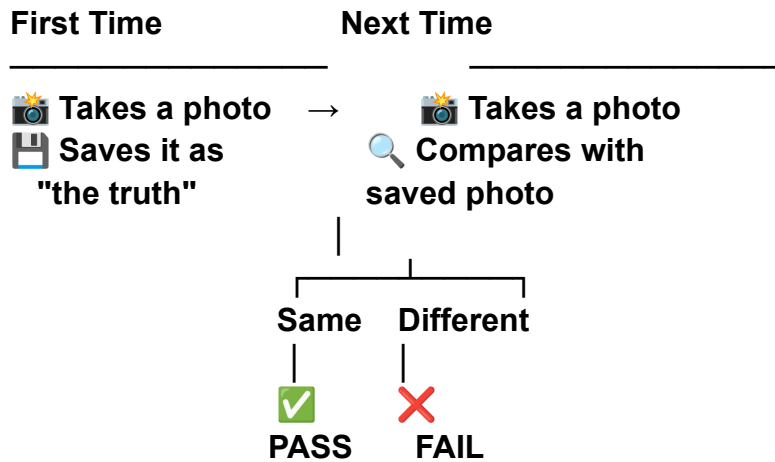
Concept	Detail
<code>toHaveScreenshot()</code>	Core method for visual comparison
Baseline	Reference image saved on first run
Diff image	Shows exactly what pixels changed
<code>maxDiffPixelRatio</code>	Tolerance for minor pixel differences
<code>mask</code>	Hide dynamic content from comparison
<code>--update-snapshots</code>	Refresh baselines after intentional UI changes
Per browser	Baselines saved separately per browser
Element vs Page	Can snapshot entire page or single element

Best practice — always disable animations, mask dynamic content like dates/ads, and set a small threshold to avoid flaky visual tests.

What Does `await expect(page).toHaveScreenshot()` Do?

Simple Explanation (Layman Terms)

Think of it like a photocopier with memory:



Step by Step What Happens

Step 1 — Takes a Screenshot

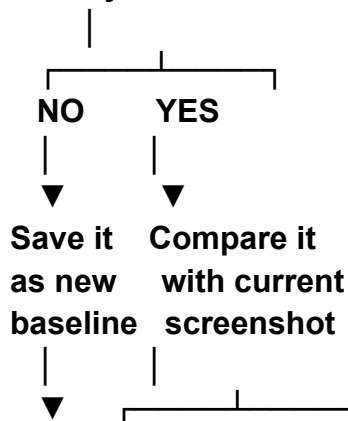
```
await expect(page).toHaveScreenshot();
```

```
//           ↑  
//    Takes a photo of the  
//    current browser page
```

- Captures exactly what the browser looks like at that moment
- Like pressing **Print Screen** on your keyboard

Step 2 — Checks for Existing Baseline

Does a baseline screenshot
already exist?



FAIL	Match	Mismatch
(first run)	✓	✗
PASS		FAIL

Step 3a — First Run (No Baseline Exists)

✗ Test FAILS on first run

📁 Baseline image is **CREATED** and saved

Why fail on first run?

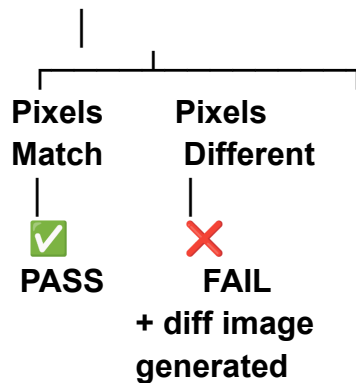
- There is nothing to compare against yet
 - Playwright creates the baseline but still fails
 - Run the test a **SECOND** time to pass
-

Step 3b — Subsequent Runs (Baseline Exists)

📷 New screenshot taken

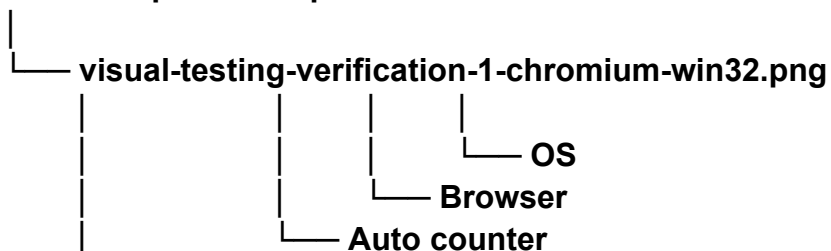


🔍 Compared pixel by pixel
with saved baseline



What Gets Saved

your-test-file.spec.ts-snapshots/



└─ Test name (auto-generated from test title)

Since no filename was given, Playwright auto-generates the name from:

- Test name
 - Screenshot counter (1, 2, 3...)
 - Browser name
 - Operating system
-

With vs Without Filename

```
// No filename — auto-generated name
await expect(page).toHaveScreenshot();
// Saves as: "visual-testing-verification-1-chromium-win32.png"
```

```
// With filename — your chosen name
await expect(page).toHaveScreenshot("MyScreenshot.png");
// Saves as: "MyScreenshot-chromium-win32.png"
```

What Happens on Failure

When pixels don't match, Playwright generates 3 images in **test-results/**:

```
test-results/
└─ visual-test-failed/
  └─ actual.png   ← What the page looks like NOW
  └─ expected.png ← What it looked like BEFORE (baseline)
  └─ diff.png     ← RED highlights showing WHAT changed
```

actual.png		expected.png		diff.png
Button [Blue]	vs	Button [Green]	=	 [Red] ← red diff

Full Lifecycle Summary

1st Run → 📸 Screenshot taken
💾 Saved as baseline
❌ Test FAILS (nothing to compare yet)

2nd Run → 📸 New screenshot taken
🔍 Compared with baseline
✅ PASSES (if page unchanged)

UI Changes → 📸 New screenshot taken
🔍 Compared with baseline
❌ FAILS (pixels are different)
📊 diff.png generated

Update Baseline → npx playwright test --update-snapshots
💾 New baseline saved
✅ Test passes going forward

Key Takeaways

Aspect	Detail
What it does	Takes screenshot and compares with saved baseline
First run	Creates baseline, test fails
Subsequent runs	Compares with baseline, passes if same
No filename	Auto-generates name from test name + browser + OS
On failure	Generates actual, expected, and diff images
Update baseline	<code>--update-snapshots</code> flag

One line summary — `toHaveScreenshot()` is Playwright's way of asking "Does the page look exactly the same as last time?"